

Session 4 · Deck 4.B1

Function Approximation and Interpolation

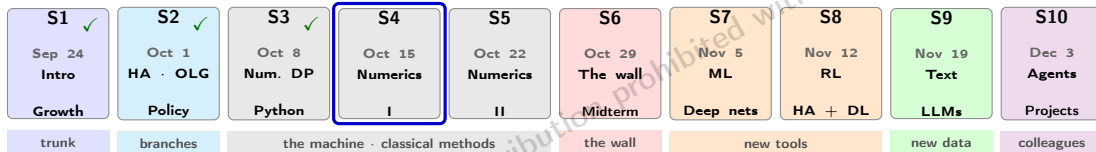
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 quadrature · 4.B3 discretizing an AR(1) · 4.B4 fast and accurate · 4.B5 the worked example, end to end.

More Accurate, and Wrong

Interpolate the policy $g(k) = \min(k, 4)$ — a household that saves everything until a constraint binds — through 21 nodes, three ways:

	max error	overshoots?
linear	0	no
cubic spline	0.043	yes
PCHIP	0.074	no

The cubic spline — the one with $O(h^4)$ accuracy — puts the policy **above a cap the model says cannot be exceeded**, and loses both monotonicity and concavity doing it.

Why this is not a small thing

Feed that interpolant back into a Bellman loop and the maximization is no longer unimodal — so 4.A2's golden section is no longer licensed, and it can converge to the wrong point.

Accuracy order is not the criterion. **The criterion is which properties survive**, because the algorithms downstream depend on them.

Why a Dynamic Program Needs This at All

$\hat{v}$ is known at N grid points and nowhere else. But:

- 4.A2's golden section asks for $\hat{v}(k')$ at points it chooses, which are not grid points;
- time iteration evaluates c_n at $k' = y - c$, which is not a grid point;
- simulation visits states that are not grid points;
- Euler errors are deliberately evaluated *off* the grid.

And the error does not stay put: an interpolation error enters $\hat{v}$, which enters the next sweep, which enters the next. **It compounds through the fixed point.**

Motivation adapted from QM Lec. 4, "Interpolation: Why?" and QM Lec. 7, "Interpolation: Recap".

Two questions, always

Local or global? Local schemes (linear, splines) use nearby nodes; global ones (polynomials, Chebyshev) use all of them, so one bad node moves the fit everywhere.

What survives? Monotonicity, concavity, and the bounds the economics imposes.

Linear: The One That Keeps Its Promises

Between k_i and k_{i+1} ,

$$\hat{v}(k) = v_i + \frac{v_{i+1} - v_i}{k_{i+1} - k_i}(k - k_i),$$

with error $O(h^2)$ and a derivative that jumps at every node.

ln k , 12 nodes	monotone	concave
linear	yes	yes
cubic spline	yes	yes
PCHIP	yes	yes

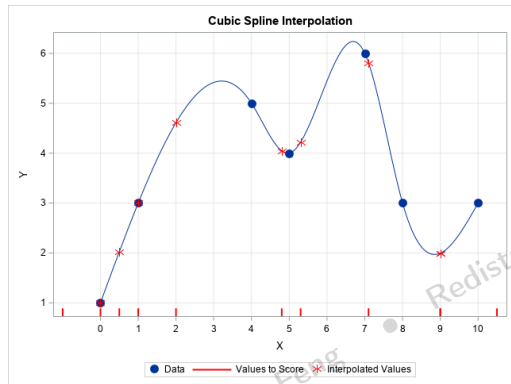
Formula from QM Lec. 7, "Piecewise Linear Interpolation"; the correction is ours, and measured.

Read the concavity row

Linear interpolation **does** preserve concavity: the chord of a concave function lies below it, so the piecewise-linear interpolant is concave — worst second difference here, 4×10^{-16} .

It matters because the golden-section argument of 4.A2 needs a concave $\hat{v}$, and **linear interpolation is what supplies it** — the humblest scheme on the table guarantees the property the method depends on.

Cubic Splines: Smooth, Accurate, and Prone to Overshoot



Cubic spline through sparse nodes.

Adapted from QM Lec. 4, "Cubic Spline" frames (figure copied in) and QM Lec. 7 F24.

A cubic on each interval, matched in value, slope and curvature at every node: C^2 overall, error $O(h^4)$.

Count the conditions. $4(N - 1)$ coefficients; $2(N - 1)$ interpolation conditions, $N - 2$ slope matches, $N - 2$ curvature matches = $4N - 6$. Two short — hence the *natural* ($\hat{v}'' = 0$ at the ends) or *clamped* boundary conditions.

The same smoothness that buys $O(h^4)$ is what makes it overshoot near a kink: the spline cannot turn sharply, so it swings past instead.

Shape-Preserving Schemes, and the Corrected Table

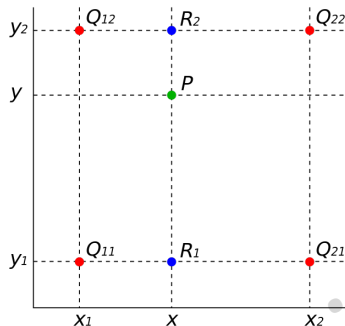
	accuracy	smooth	monotone	concavity
Linear	$O(h^2)$	C^0	yes	yes
Cubic spline	$O(h^4)$	C^2	no	no
PCHIP (Fritsch–Carlson)	$O(h^3)$	C^1	yes	no
Schumaker	$O(h^2)$	C^1	yes	yes

Schumaker's quadratic spline is the only row that preserves both, which is why it is the standard choice when a value function must stay concave.

The published version of this table marks Linear as “concavity: no”. Our measurement says otherwise, and the corrected entry is in bold. **Recompute the tables in your sources.**

Structure from QM Lec. 7, “Interpolation: Comparison” and “Shape-Preserving Interpolation”, with the linear/concavity entry corrected.

More Than One Dimension



Bilinear interpolation on a rectangular cell.

Bilinear: interpolate along x on both edges, then along y between the two results. Exact for $1, x, y, xy$; the natural extension of linear to a rectangular grid, and it keeps monotonicity in each argument separately.

Triangular (barycentric): split each cell into triangles and use weights $w_1, w_2, w_3 \geq 0$ summing to one. The interpolant is a genuine convex combination of the three vertices, so it **cannot overshoot** — and it works on unstructured grids.

The cost is the real problem: a tensor grid needs N^d points, which is 3.A3's curse arriving in the interpolation step rather than the maximization.

Adapted from QM Lec. 4, "Bilinear" and "Triangular Interpolation" frames; the bilinear figure is copied in and renamed (the source filename contains a dot and breaks SVG-aware drivers).

Smolyak: Most of a Tensor Grid Is Wasted

A tensor grid takes every combination of one-dimensional nodes, so it spends points on corners where *every* coordinate is extreme at once — states the model almost never visits.

Smolyak's construction keeps only the combinations whose total resolution is modest. With one-dimensional node sets $G^1, G^2, \dots$ indexed by $i_1, \dots, i_d$, take the union of tensor products over a *budget* on the index sum,

$$\Phi(q, d) = \bigcup_{q-d+1 \leq |i| \leq q} (G^{i_1} \times \dots \times G^{i_d}),$$

writing $|i| = \sum_j i_j$ and $q = d + \mu$. High resolution in *one* dimension is allowed; high resolution in all of them at once is not. μ is the **level**: $\mu = 0$ is a single point.

Smolyak (1963); in *economics* Krueger & Kubler (2004), Malin, Krueger & Kubler (2011), Judd, Maliar, Maliar & Valero (2014). Our run reproduces

Malin–Krueger–Kubler's own table at $\mu = 2$ exactly: $d = 2, 4, 5, 12 \Rightarrow 13, 41, 61, 313$.

d	Smolyak $\mu=2$	5^d	9^d
2	13	25	81
3	25	125	729
4	41	625	6,561
5	61	3,125	59,049
10	221	9.8 million	3.5×10^9
12	313	244 million	2.8×10^{11}

At ten dimensions the sparse grid needs **221 points** where a five-point tensor grid needs nearly ten million — a factor of **44,000**.

The Ladder: Nested Node Sets, and Why Nesting Is the Point

The one-dimensional sets are not arbitrary. Take

$$m_1 = 1, \quad m_i = 2^{i-1} + 1,$$

and let G^i be the m_i **extrema** of a Chebyshev polynomial,

$$x_j^i = -\cos\left(\frac{\pi(j-1)}{m_i-1}\right), \quad j = 1, \dots, m_i,$$

with $G^1 = \{0\}$ by convention.

i	m_i	G^i
1	1	$\{0\}$
2	3	$\{-1, 0, 1\}$
3	5	$\{-1, -0.7071, 0, 0.7071, 1\}$
4	9	$\{\pm 1, \pm 0.9239, \pm 0.7071, \pm 0.3827, 0\}$

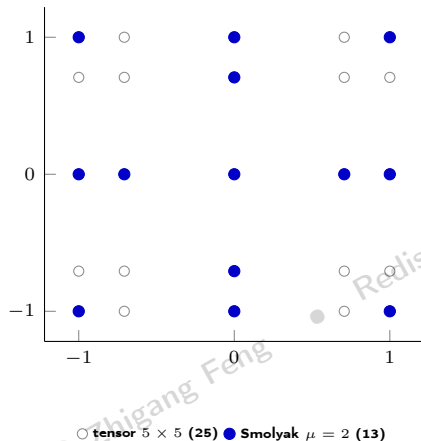
$$G^1 \subset G^2 \subset G^3 \subset \dots$$

Checked in the run: every level contains the one below it. That is the property the whole construction rests on.

Nesting buys two things. The union that defines Φ is mostly **overlap**, so the point count grows far more slowly than the number of index sets; and **refining the grid reuses every function value you already paid for** — going from μ to $\mu + 1$ adds points, it does not move them.

Equally spaced nodes are *not* nested this way, and Chebyshev *zeros* are not either. The extrema are chosen for exactly this reason.

Building It in Two Dimensions, By Hand



$\mu = 0$: only $i_1 + i_2 \leq 2$, so $(1, 1)$ alone and $\Phi = \{(0, 0)\}$ — one point.

$\mu = 1$: $2 \leq i_1 + i_2 \leq 3$ admits $(1, 1)$, $(1, 2)$, $(2, 1)$, and the union is the five-point cross

$$\{(0, 0), (\pm 1, 0), (0, \pm 1)\}.$$

$\mu = 2$: adds $(1, 3)$, $(2, 2)$, $(3, 1)$ — **13** of the 25 tensor points.

Notice *which* 12 are dropped: every point with two non-central coordinates except the four corners. The sparse grid keeps the **axes and the corners** and throws away the interior cross-terms.

Three Dimensions, and the Signed Combination

At $d = 3$, $\mu = 2$ the admissible multi-indices are the ten

(1, 1, 1) (1, 1, 2) (1, 2, 1)
 (2, 1, 1) (1, 1, 3) (1, 2, 2)
 (1, 3, 1) (2, 1, 2) (2, 2, 1)
 (3, 1, 1)

whose union is **25 distinct points**, against $5^3 = 125$.

The approximation is *not* a plain interpolation on those points. It is a signed combination of tensor interpolants,

$$\hat{f} = \sum_{q-d+1 \leq |i| \leq q} (-1)^{q-|i|} \binom{d-1}{q-|i|} (U^{i_1} \otimes \dots \otimes U^{i_d}),$$

which cancels the overlaps the union created.

Coefficients at $d = 2$, $\mu = 2$ ($q = 4$)

i	$ i $	$(-1)^{q- i } \binom{d-1}{q- i }$
(1, 1)	2	0
(1, 2), (2, 1)	3	-1
(1, 3), (2, 2), (3, 1)	4	+1

The zero explains why the two index ranges you see in the literature — $d \leq |i| \leq q$ and $q - d + 1 \leq |i| \leq q$ — give the same answer: the extra terms have coefficient $\binom{d-1}{q-|i|} = 0$, and by nesting they add no points either.

Signed weights: a sparse *quadrature* rule built this way can have negative ones, so it need not be positive — unlike Gauss rules (4.B2).

Construction and the $d = 3$ list from Malin, Krueger & Kubler (2011). Their example frame labels it $\Phi(5, 2)$; with $d = 3$, $\mu = 2$ it is $\Phi(5, 3)$.

What Sparsity Costs, and Where It Reappears

Nothing is free. The sparse grid is exact for a **smaller** set of polynomials than the tensor grid it replaces — roughly the *total-degree* set rather than the full tensor set — so it loses the cross-terms of highest order. For $f \in C^k$ the error is bounded by

$$C_d n^{-k} (\log n)^{(d-1)(k+1)} \|f\|_k,$$

$n = |\Phi(q, d)|$: the tensor rate n^{-k} , spoiled by a log factor that grows with d . **The curse is weakened, not broken.**

That is usually the right trade in economics, because the functions we approximate are smooth and their high-order interactions are small. It is the wrong trade when the object has a **kink** whose position depends on several states at once — a borrowing constraint that binds only for the young and unlucky.

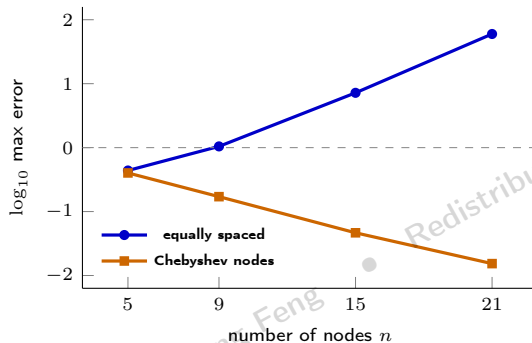
Two uses, one construction

Here Smolyak is a **node set** — where to evaluate a function you are interpolating, and where to place quadrature nodes.

In **Session 5** it becomes a **basis**: the same index rule selects which polynomials enter a projection method, and the coefficients are chosen to kill a residual.

Same combinatorics, different job. Worth keeping the distinction straight, because the literature uses “Smolyak” for both.

Global Polynomials, and Why Equally Spaced Nodes Fail



Interpolate Runge's function $1/(1 + 25x^2)$ on $[-1, 1]$ by a single polynomial through n nodes.

n	equispaced	Chebyshev
5	0.44	0.40
9	1.05	0.17
15	7.19	0.047
21	59.8	0.015

More points makes the equispaced fit **worse**, by a factor of 136 between $n = 5$ and $n = 21$. The oscillation is at the ends, and it diverges.

Chebyshev Nodes: Cluster at the Ends

On $[-1, 1]$,

$$x_i = \cos\left(\frac{2i-1}{2n}\pi\right), \quad i = 1, \dots, n,$$

mapped to $[a, b]$ by $s_i = \frac{a+b}{2} + \frac{b-a}{2}x_i$. These nodes **minimise the maximum** interpolation error over all node choices, and they are dense exactly where the equispaced fit blew up — at the boundaries.

At $n = 21$ the Chebyshev error is 0.015 against the equispaced 59.8: **four thousand times better**, with the same number of points and the same degree.

Read the formula carefully

$i = 1$ gives $\cos(\pi/2n)$, which is close to $+1$.
The nodes come out **descending**.

Every routine downstream — a monotonicity ratchet, a bracketing search, `np.interp` — assumes an ascending grid. **Sort them**.

This is the one line of this deck that students reliably get wrong the first time they code it, and the failure is silent.

Node formula and minimax property from QM Lec. 7, “Chebyshev Nodes”; the ordering warning is ours.

Where Polynomials Cannot Be Rescued: Gibbs

Runge's problem is fixed by moving the nodes. A genuine **discontinuity** is not.

Fit a degree-20 polynomial to a step function and it overshoots by 0.41 — on a jump of size one. Raise the degree and the overshoot *does not shrink*; it merely moves closer to the jump. That is the Gibbs phenomenon, and it is a property of the approximating family, not of the fit.

What to do instead

Use a **local** scheme, or split the domain at the kink and approximate each piece separately.

Economics produces kinks routinely: borrowing constraints, occasionally binding bounds, discrete choices.

A neural network is also a global approximator — but with a *kinked* basis, which is why ReLU networks handle these problems more gracefully (S7).

Gibbs discussion adapted from QM Lec. 5, "Case Study: Fitting Data"; overshoot measured here.

Takeaways

- Interpolation is not a detail of a dynamic program — it is inside the fixed point, so its error **compounds through every sweep**.
- **Linear interpolation preserves concavity**, contrary to the table everyone prints, and that is precisely what licenses golden section inside the Bellman loop.
- Cubic splines buy $O(h^4)$ and lose shape. On a capped policy the spline **overshot the cap by 0.043** and broke both monotonicity and concavity; linear interpolation was exact.
- Equally spaced nodes make a global polynomial **worse** as you add points — error 0.44 to 59.8 from $n = 5$ to 21. Chebyshev nodes fix it; at $n = 21$ they are four thousand times better.
- $x_i = \cos((2i - 1)\pi/2n)$ comes out **descending**. Sort it.
- A discontinuity cannot be fixed by moving nodes. **Gibbs does not go away** — go local, or split the domain.
- In several dimensions a tensor grid is mostly waste. **Smolyak** needs 221 points in ten dimensions where a five-point tensor grid needs 9.8 million — and it returns in Session 5 as a *basis* rather than a node set.

Bridge: The Other Thing a Grid Cannot Do

We can now represent $\hat{v}$ between grid points and know which schemes keep the properties the algorithms need.

But the Bellman equation does not only evaluate $\hat{v}$ — it takes an **expectation** of it:

$$\mathbb{E}[\hat{v}(k', z') \mid z] = \int \hat{v}(k', z') dF(z' \mid z).$$

Every solve so far has replaced that integral with a finite sum $\sum_m \Pi_{jm} \hat{v}(k', z_m)$ — and we have written Π down by hand twice now, without ever saying where it comes from.

Next (4.B2)

Quadrature: how an integral becomes a finite sum, and how much accuracy the choice of rule is worth.

Then **4.B3** answers the question directly — where Π comes from, and which of the two standard constructions you should actually use.